

```
pip install deap
```

Requirement already satisfied: deap in c:\users\kamal\anaconda3\envs\myenv\lib\site-packages (1.4.1)

Requirement already satisfied: numpy in c:\users\kamal\anaconda3\envs\myenv\lib\site-packages (from deap) (1.24.3)

Note: you may need to restart the kernel to use updated packages.

```
%matplotlib inline
```

```
import os
```

```
seed = 42
```

```
os.environ['PYTHONHASHSEED']=str(seed)
```

```
import pandas as pd
```

```
import numpy as np
```

```
import random
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns; sns.set()
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_squared_error, r2_score
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import GroupShuffleSplit
```

```
import tensorflow as tf
```

```
from tensorflow.keras import Sequential
```

```
from tensorflow.keras.layers import LSTM, Dense, Dropout, Masking, TimeDistributed
```

```
from keras.models import Sequential
```

```
from keras.layers import Dense, Conv1D
```

```
from keras.layers import Dropout, Input
```

```
from keras import initializers
```

```
from keras.layers import MaxPooling1D
```

```
from keras.models import Model
```

```
from keras.layers import TimeDistributed, Flatten
```

```
from keras.layers import Dense, Activation, Dropout
```

```
from keras.callbacks import EarlyStopping
```

```
from keras.callbacks import ModelCheckpoint
```

```
import keras
```

```
from tensorflow import keras
```

```
from tensorflow.keras import layers
```

```
from deap import algorithms, base, creator, tools
```

```
from tensorflow.keras.optimizers import Adam
```

```
dir_path = 'C:\\Users\\Kamal\\Documents\\AUC\\thesiswork\\env_eweda\\CMaps'
```

```
os.chdir(dir_path)
```

```
print(os.getcwd())
```


3	1585.61	1392.27	14.62	...	522.49	2388.08	8146.56	8.3878
0.03	392							
4	1588.63	1397.65	14.62	...	522.58	2388.03	8147.80	8.3869
0.03	392							

	s_18	s_19	s_20	s_21
0	2388	100.0	39.11	23.3537
1	2388	100.0	38.99	23.4491
2	2388	100.0	38.85	23.3669
3	2388	100.0	38.96	23.2951
4	2388	100.0	39.14	23.4583

[5 rows x 26 columns]

```
def add_remaining_useful_life(df):
    # Get the total number of cycles for each unit
    grouped_by_unit = df.groupby(by="unit_nr")
    max_cycle = grouped_by_unit["time_cycles"].max()

    # Merge the max cycle back into the original frame
    result_frame = df.merge(max_cycle.to_frame(name='max_cycle'),
left_on='unit_nr', right_index=True)

    # Calculate remaining useful life for each row
    remaining_useful_life = result_frame["max_cycle"] -
result_frame["time_cycles"]
    result_frame["RUL"] = remaining_useful_life

    # drop max_cycle as it's no longer needed
    result_frame = result_frame.drop("max_cycle", axis=1)
    return result_frame
```

```
train = add_remaining_useful_life(train)
train[index_names+['RUL']].head()
```

	unit_nr	time_cycles	RUL
0	1	1	258
1	1	2	257
2	1	3	256
3	1	4	255
4	1	5	254

```
X_train = train[setting_names + sensor_names].copy()
y_train = train['RUL'].copy()
y_train_clipped = y_train.clip(upper=125)
```

```
X_test = test.drop('time_cycles',
axis=1).groupby('unit_nr').last().copy() # get last row of each engine
def evaluate(y_true, y_hat, label='test'):
    mse = mean_squared_error(y_true, y_hat)
    rmse = np.sqrt(mse)
```

```

    variance = r2_score(y_true, y_hat)
    print('{} set RMSE: {}, R2: {}'.format(label, rmse, variance))
lm = LinearRegression()
lm.fit(X_train, y_train_clipped)

# predict and evaluate
y_hat_train = lm.predict(X_train)
evaluate(y_train_clipped, y_hat_train, 'train')

y_hat_test = lm.predict(X_test)
evaluate(y_test, y_hat_test)

train set RMSE:19.329629974927535, R2:0.7736229075130826
test set RMSE:22.288215837615585, R2:0.7101043061544098

# before we start plotting, lets implement the condition specific scaling
def add_operating_condition(df):
    df_op_cond = df.copy()

    df_op_cond['setting_1'] = df_op_cond['setting_1'].round()
    df_op_cond['setting_2'] =
df_op_cond['setting_2'].round(decimals=2)

    # converting settings to string and concatenating makes the operating condition into a categorical variable
    df_op_cond['op_cond'] = df_op_cond['setting_1'].astype(str) + '_'
+ \
                                df_op_cond['setting_2'].astype(str) + '_' + \
                                df_op_cond['setting_3'].astype(str)

    return df_op_cond

def condition_scaler(df_train, df_test, sensor_names):
    # apply operating condition specific scaling
    scaler = StandardScaler()
    for condition in df_train['op_cond'].unique():
        scaler.fit(df_train.loc[df_train['op_cond']==condition,
sensor_names])
        df_train.loc[df_train['op_cond']==condition, sensor_names] =
scaler.transform(df_train.loc[df_train['op_cond']==condition,
sensor_names])
        df_test.loc[df_test['op_cond']==condition, sensor_names] =
scaler.transform(df_test.loc[df_test['op_cond']==condition,
sensor_names])
    return df_train, df_test
X_train_condition = add_operating_condition(train)
X_test_condition = add_operating_condition(X_test)

X_train_condition_scaled, X_test_condition_scaled =

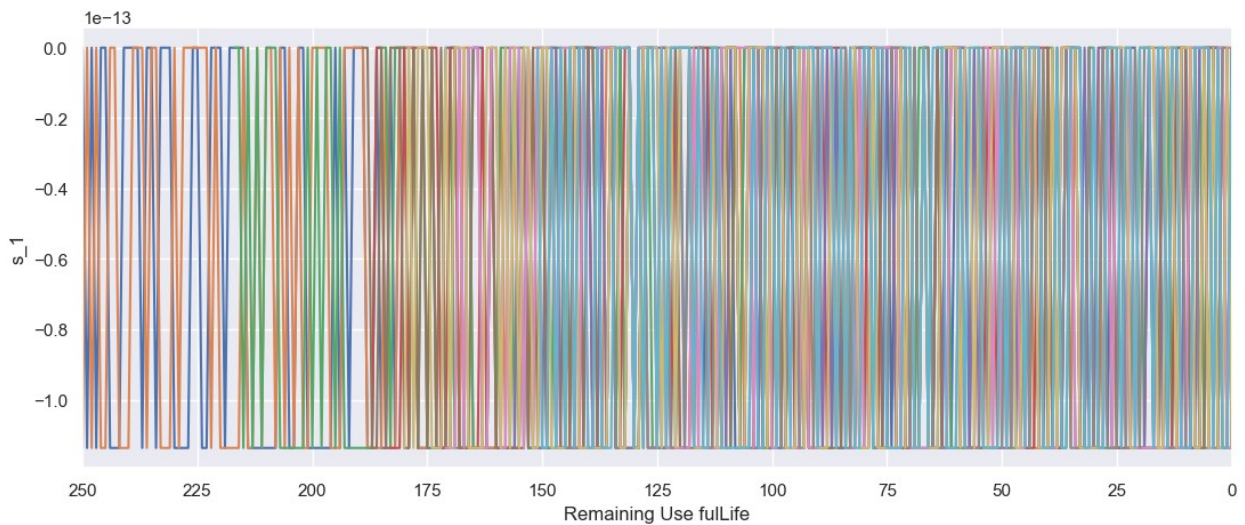
```

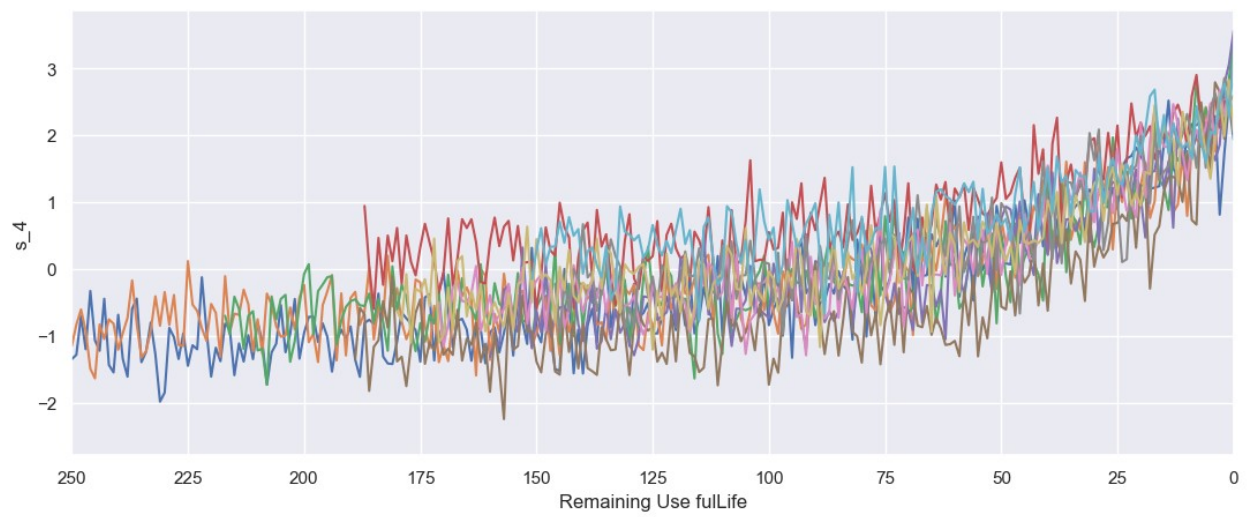
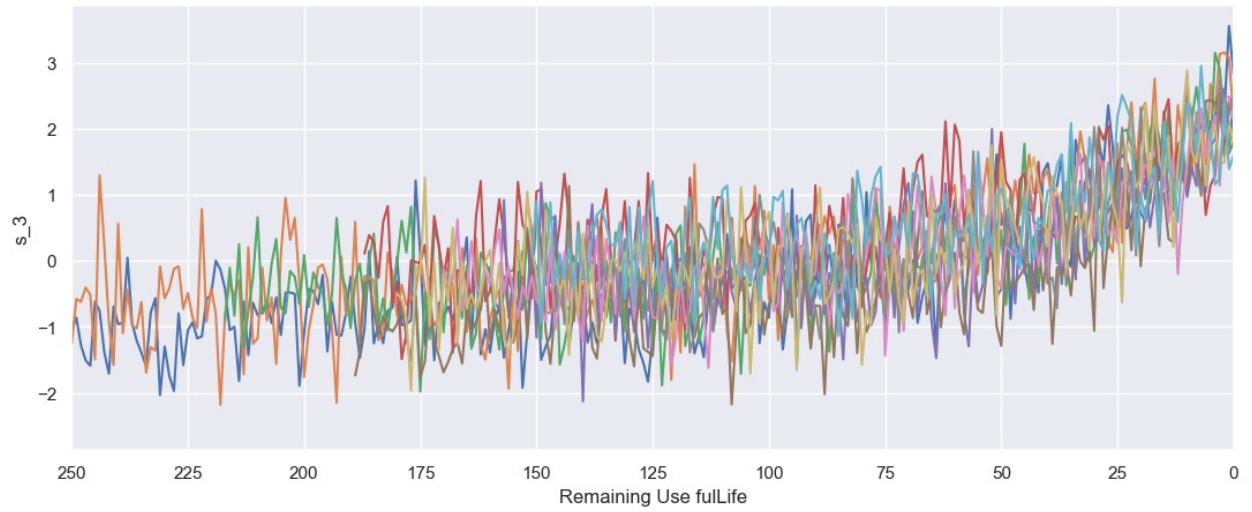
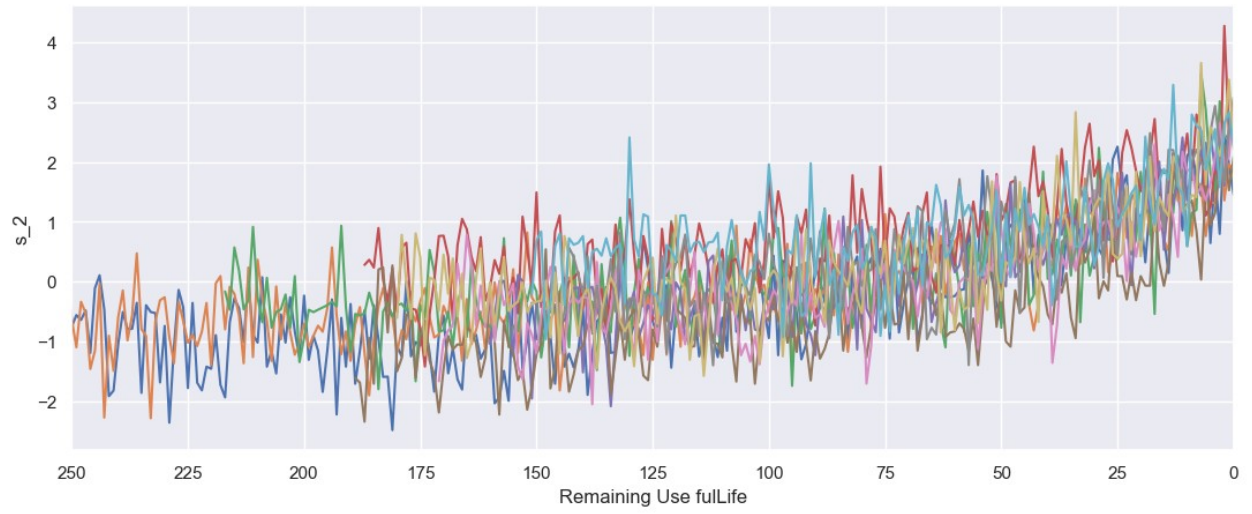
```

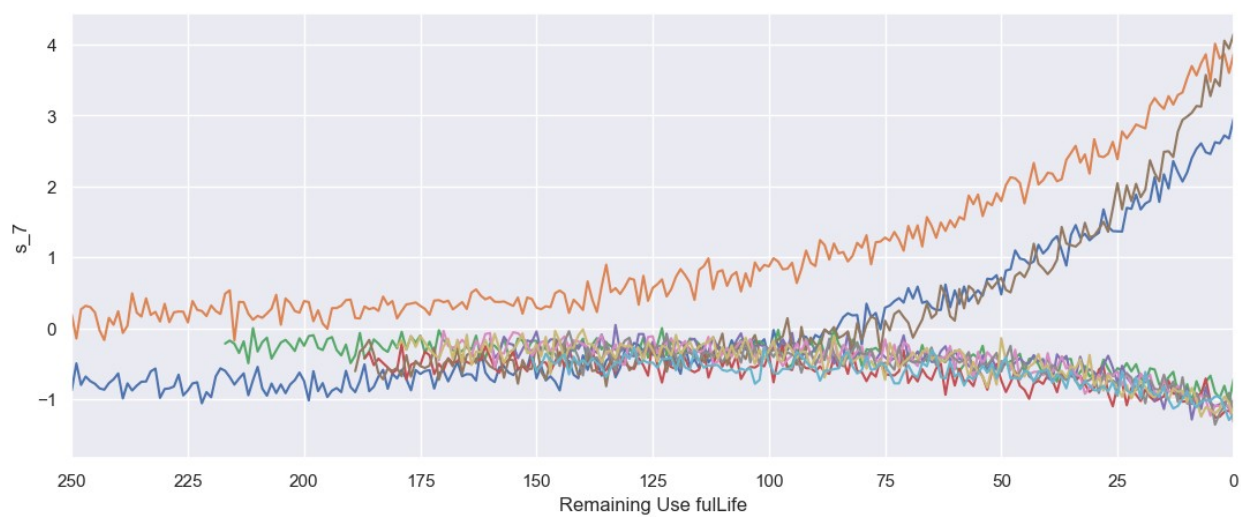
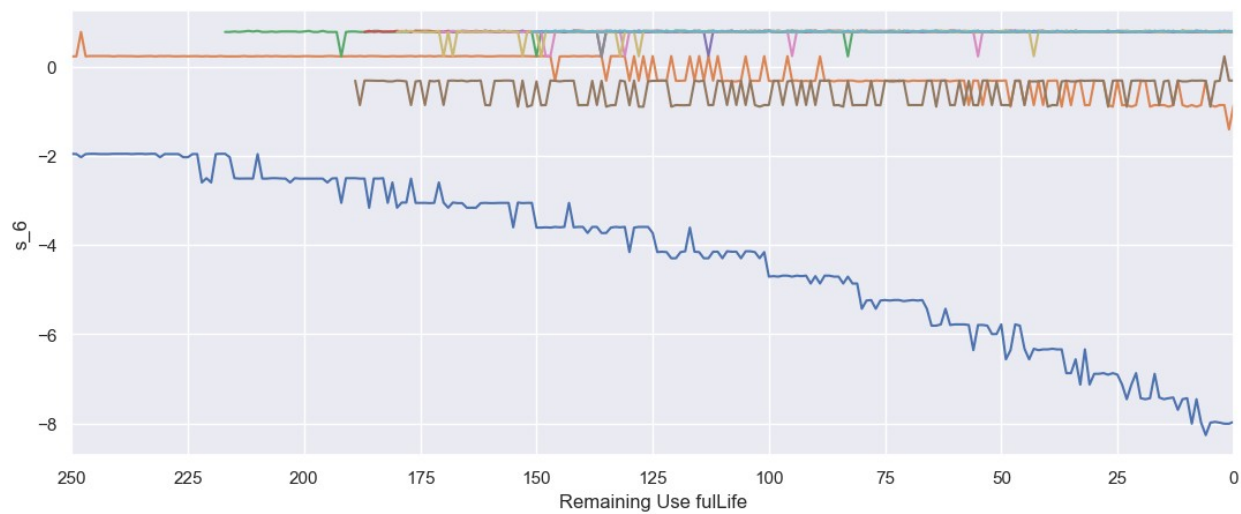
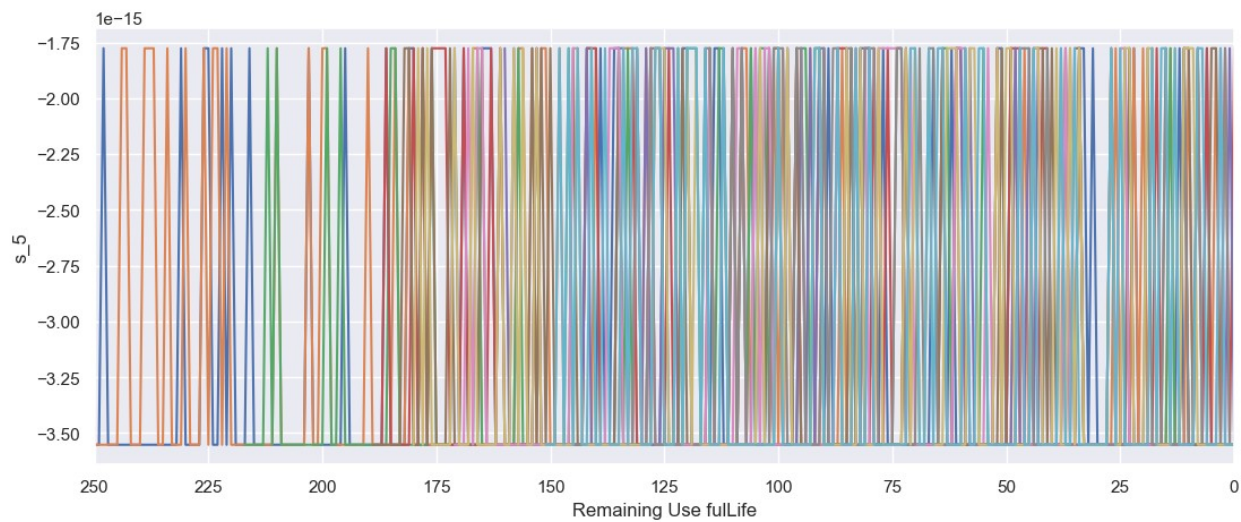
condition_scaler(X_train_condition, X_test_condition, sensor_names)
def plot_signal(df, signal_name, unit_nr=None):
    plt.figure(figsize=(13,5))

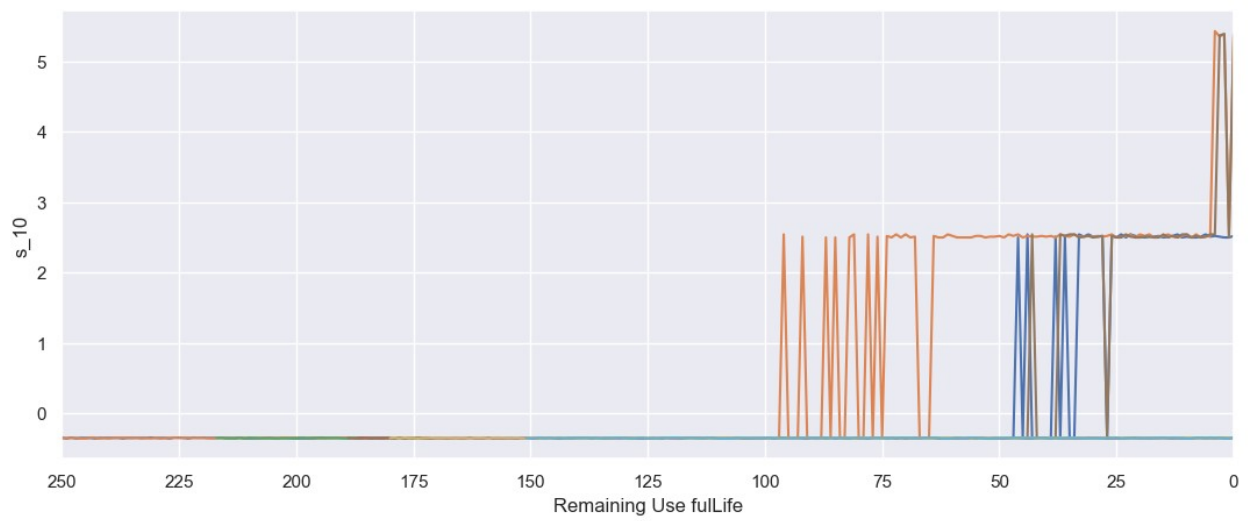
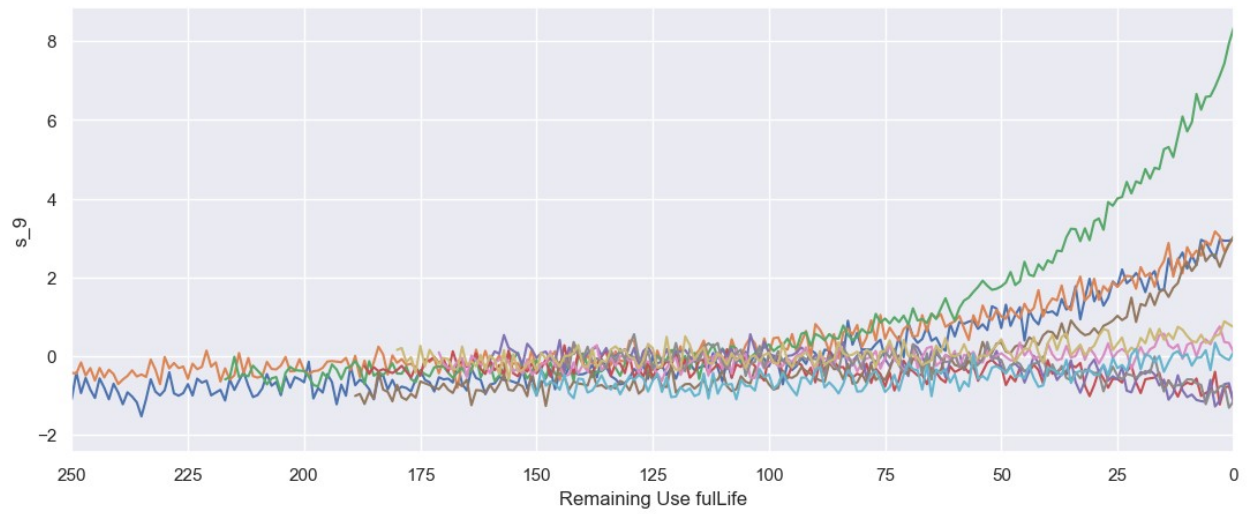
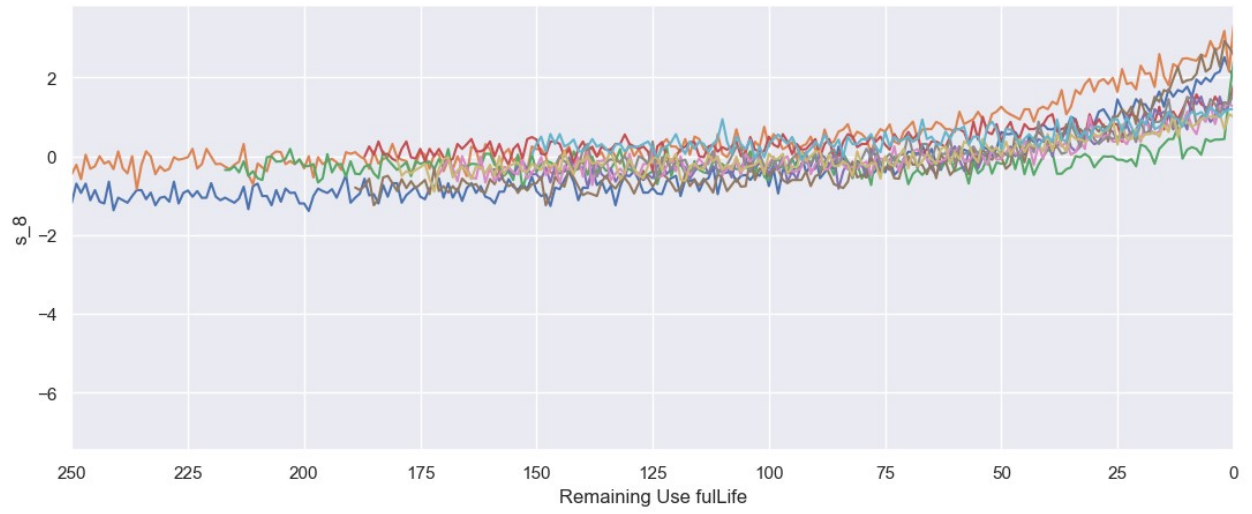
    if unit_nr:
        plt.plot('RUL', signal_name,
                 data=df[df['unit_nr']==unit_nr])
    else:
        for i in train['unit_nr'].unique():
            if (i % 10 == 0): # only plotting every 10th unit_nr
                plt.plot('RUL', signal_name,
                         data=df[df['unit_nr']==i])
    plt.xlim(250, 0) # reverse the x-axis so RUL counts down to zero
    plt.xticks(np.arange(0, 275, 25))
    plt.ylabel(signal_name)
    plt.xlabel('Remaining Use fullLife')
    plt.show()
for sensor in sensor_names:
    plot_signal(X_train_condition_scaled, sensor)

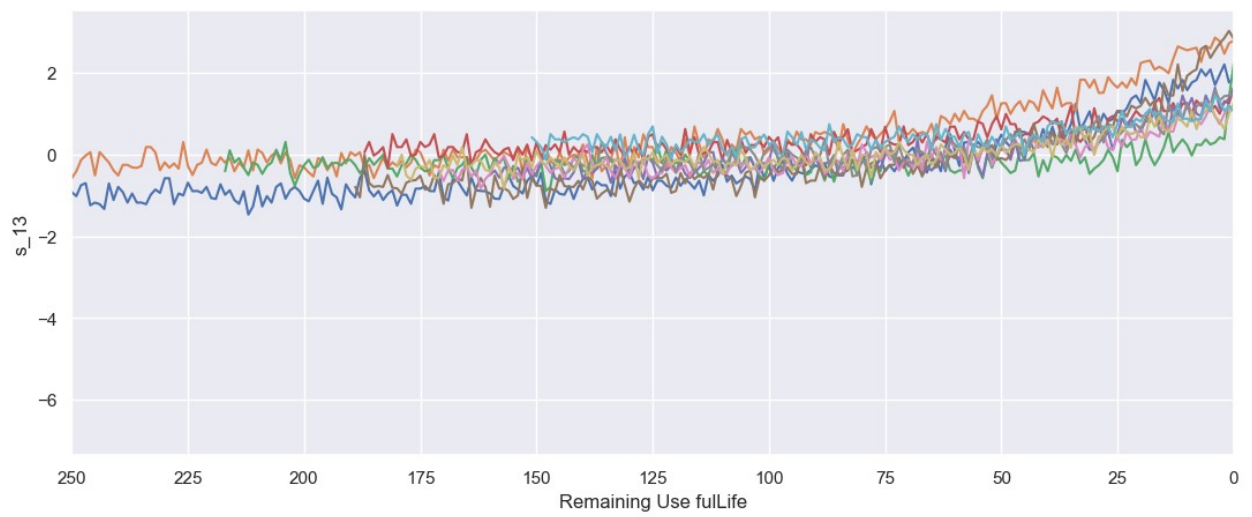
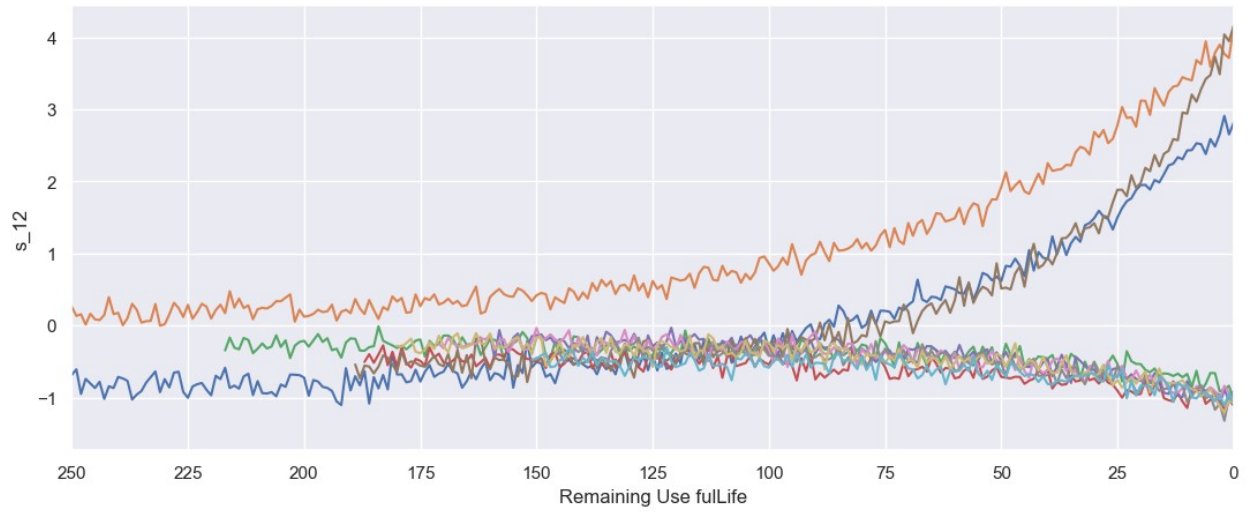
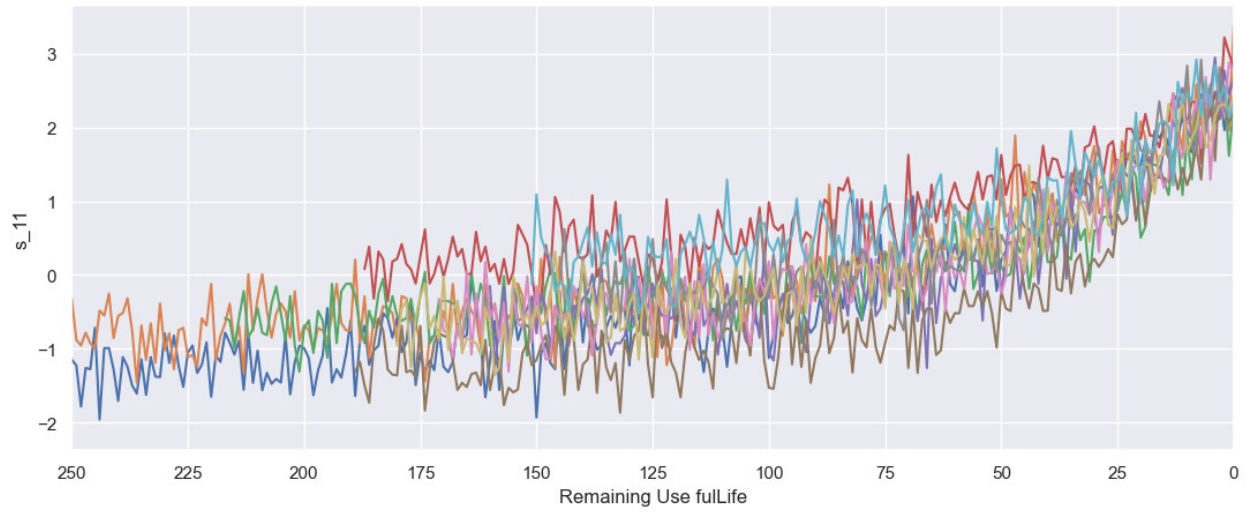
```

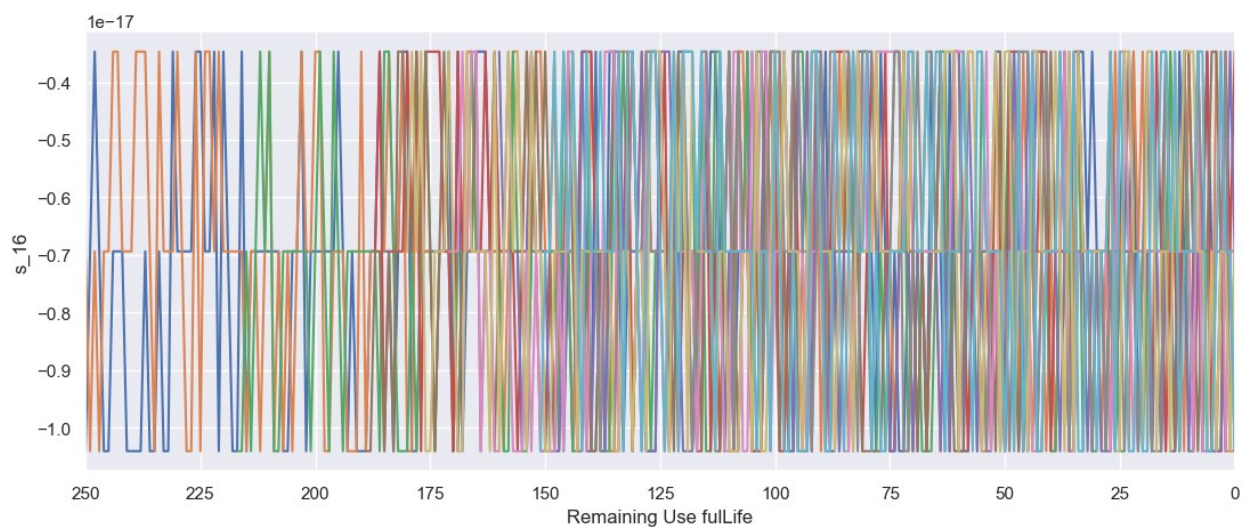
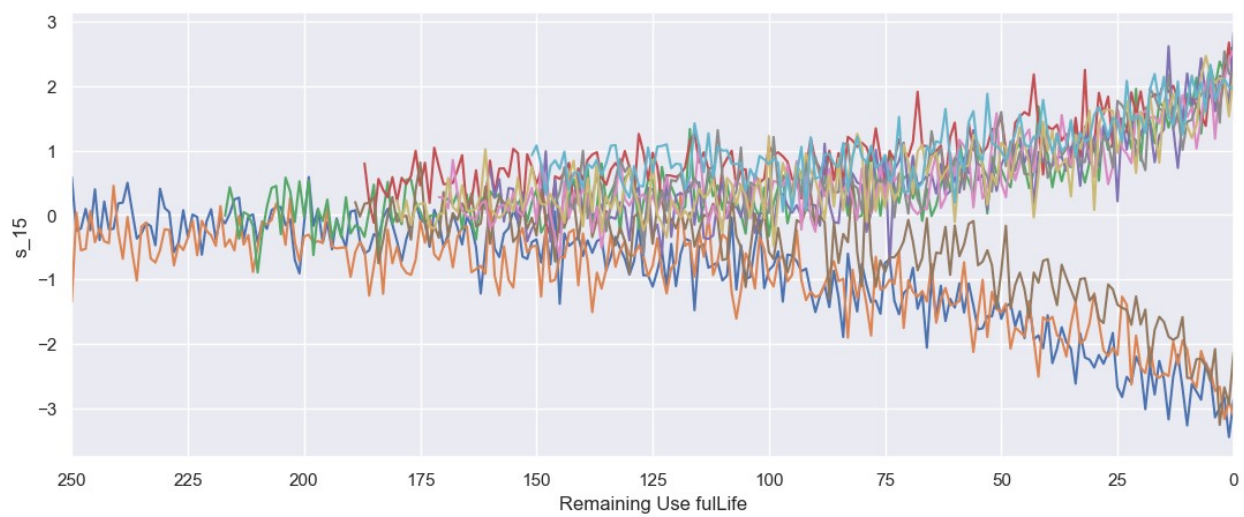
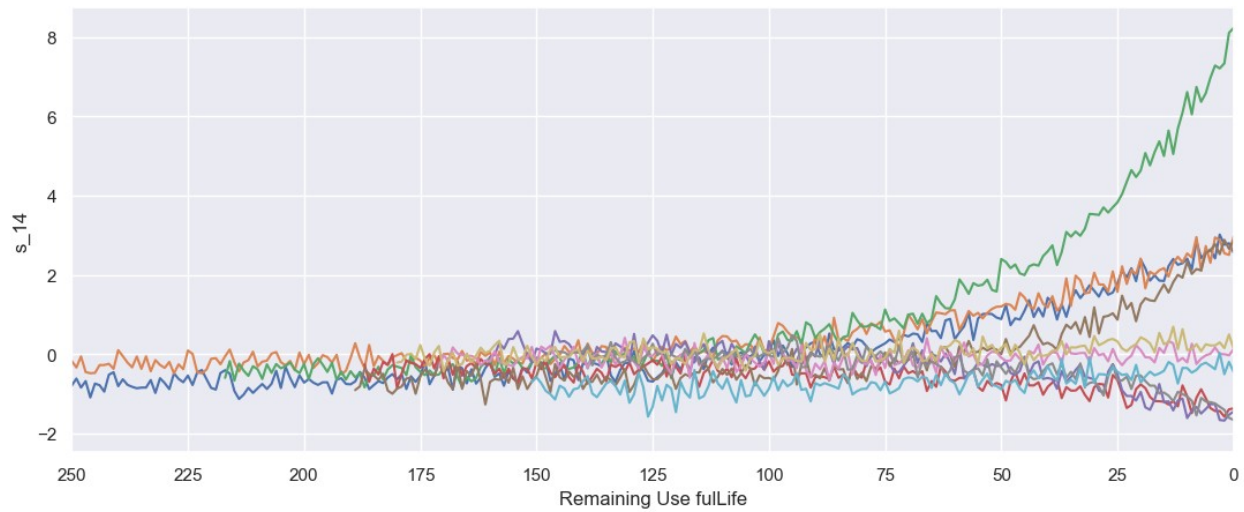


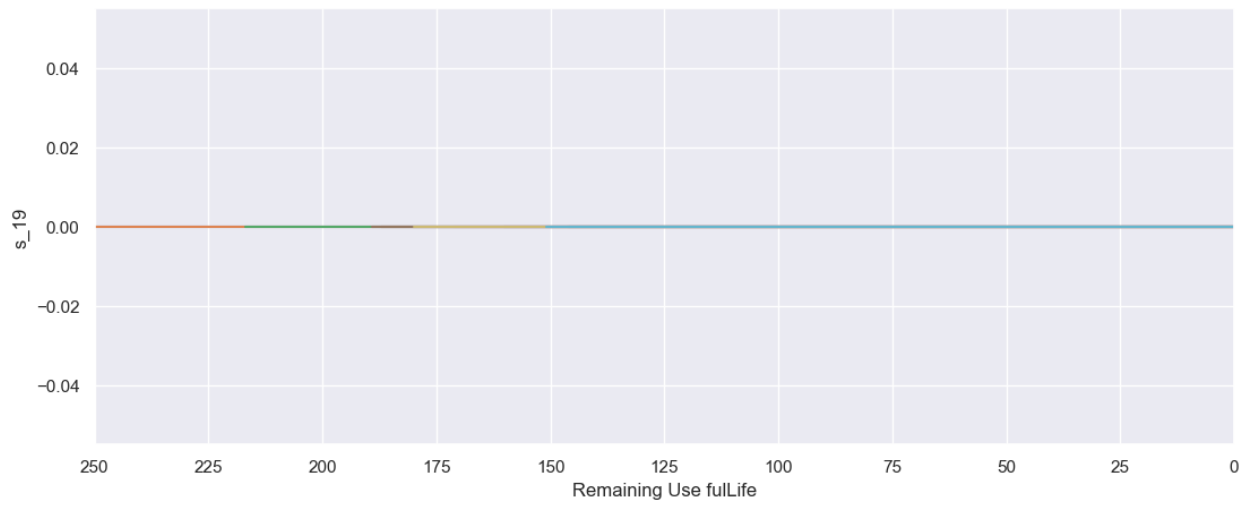
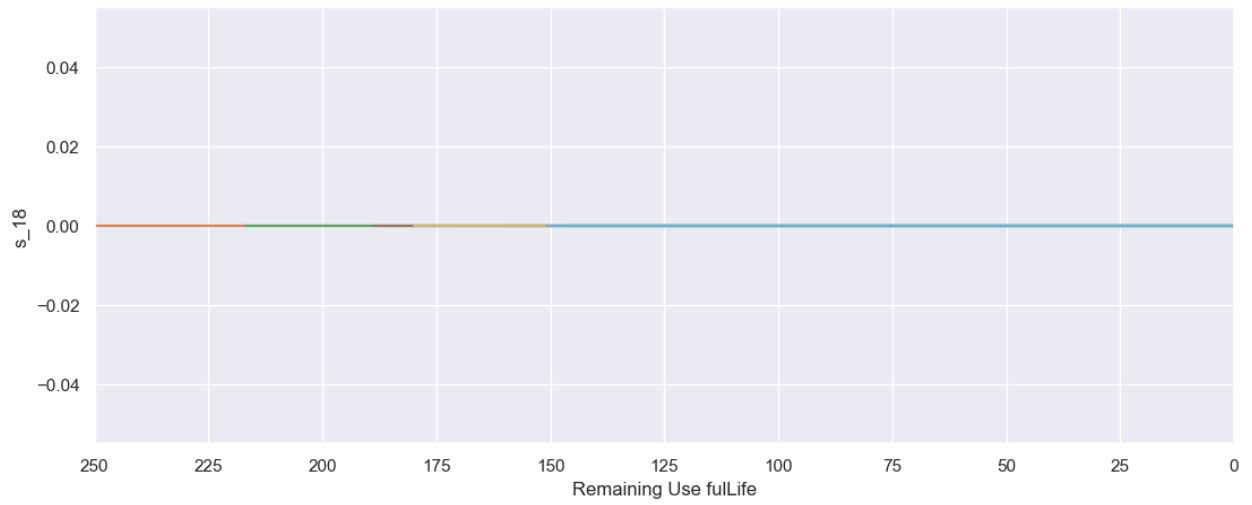
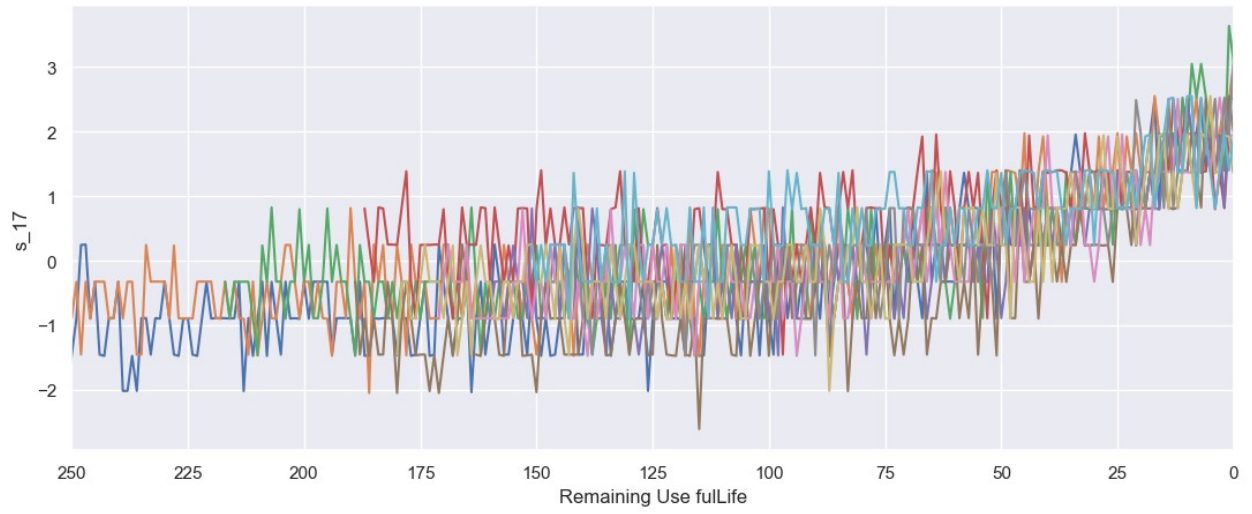


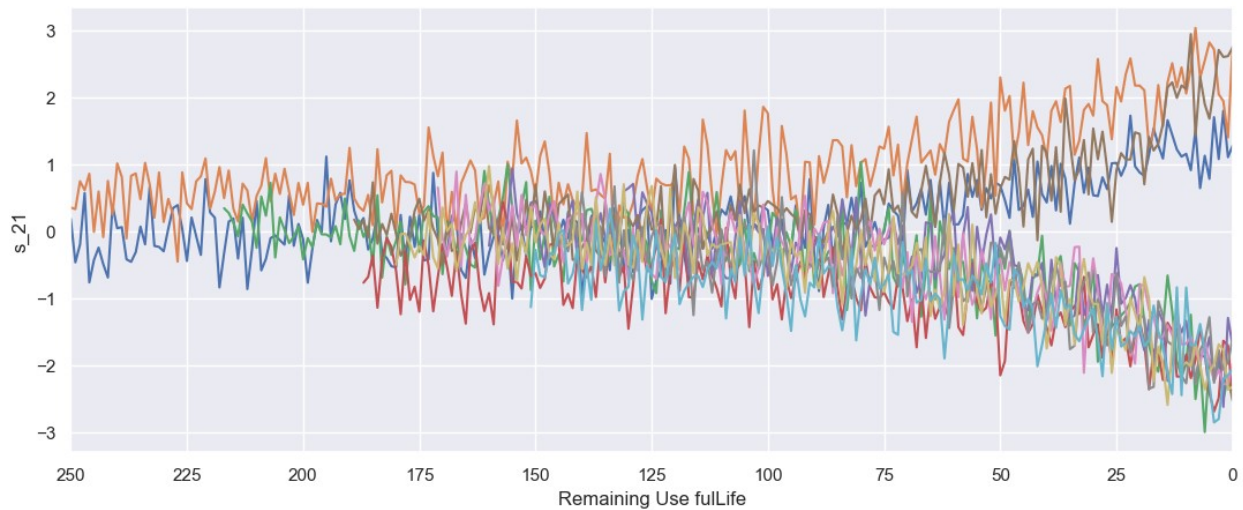
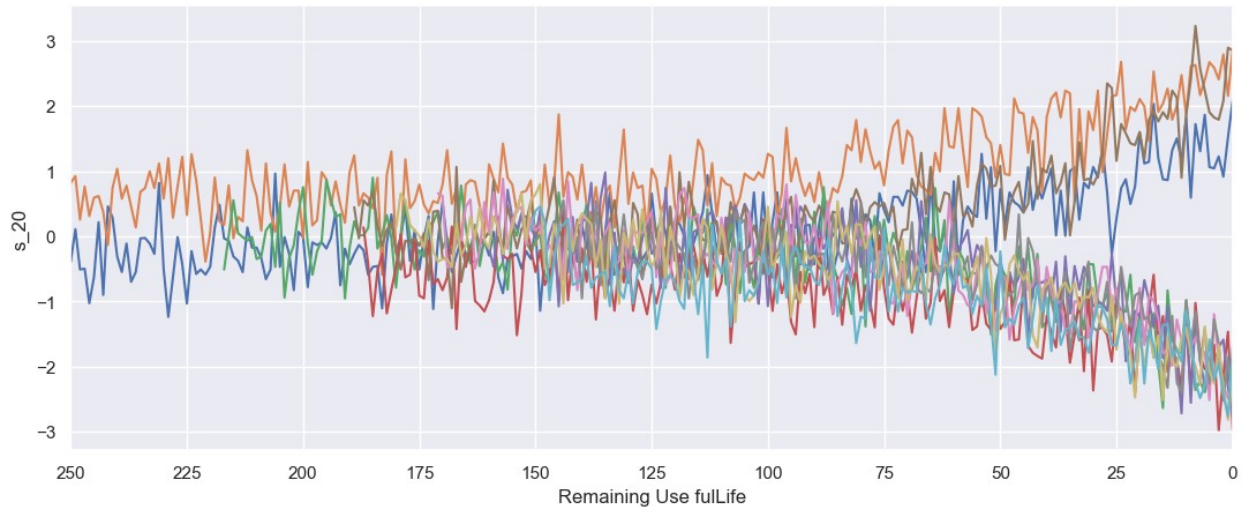












```
remaining_sensors = ['s_2', 's_3', 's_4', 's_7', 's_8', 's_9',
                    's_11', 's_12', 's_13', 's_14', 's_15', 's_17', 's_20', 's_21']
drop_sensors = [element for element in sensor_names if element not in
remaining_sensors]

def exponential_smoothing(df, sensors, n_samples, alpha=0.4):
    df = df.copy()
    # first, take the exponential weighted mean
    df[sensors] = df.groupby('unit_nr')[sensors].apply(lambda x:
x.ewm(alpha=alpha).mean())

    # second, drop first n_samples of each unit_nr to reduce filter
    delay
    def create_mask(data, samples):
        result = np.ones_like(data)
        result[0:samples] = 0
        return result
```

```

    mask = df.groupby('unit_nr')['unit_nr'].transform(create_mask,
samples=n_samples).astype(bool)
    df = df[mask]

    return df
gss = GroupShuffleSplit(n_splits=1, train_size=0.80, random_state=42)
# eventhough we set np and tf seeds, gss requires its own seed

def train_val_group_split(X, y, gss, groups, print_groups=True):
    for idx_train, idx_val in gss.split(X, y, groups=groups):
        if print_groups:
            print('train_split_engines', train.iloc[idx_train]
['unit_nr'].unique())
            print('validate_split_engines', train.iloc[idx_val]
['unit_nr'].unique(), '\n')

            X_train_split = X.iloc[idx_train].copy()
            y_train_split = y.iloc[idx_train].copy()
            X_val_split = X.iloc[idx_val].copy()
            y_val_split = y.iloc[idx_val].copy()
        return X_train_split, y_train_split, X_val_split, y_val_split

d = {'unit_nr': [1]*5 + [2]*5,
      'X': [10., 10.2, 10.4, 10.6, 10.8, 20., 20.2, 20.4, 20.6, 20.8],
      'y': [1.4, 1.4, 1.3, 1.2, 1.1, 2.4, 2.4, 2.3, 2.2, 2.1]}
example_df = pd.DataFrame(data=d)
example_df

```

	unit_nr	X	y
0	1	10.0	1.4
1	1	10.2	1.4
2	1	10.4	1.3
3	1	10.6	1.2
4	1	10.8	1.1
5	2	20.0	2.4
6	2	20.2	2.4
7	2	20.4	2.3
8	2	20.6	2.2
9	2	20.8	2.1

```

def gen_train_data(df, sequence_length, columns):
    data = df[columns].values
    num_elements = data.shape[0]

    # -1 and +1 because of Python indexing
    for start, stop in zip(range(0, num_elements-(sequence_length-1)),
range(sequence_length, num_elements+1)):
        yield data[start:stop, :]
gen = gen_train_data(example_df[example_df['unit_nr']==1],

```

```

sequence_length=4, columns=['X'])
list(gen)

[array([[10. ],
        [10.2],
        [10.4],
        [10.6]]),
 array([[10.2],
        [10.4],
        [10.6],
        [10.8]])]

def gen_data_wrapper(df, sequence_length, columns,
unit_nrs=np.array([])):
    if unit_nrs.size <= 0:
        unit_nrs = df['unit_nr'].unique()

        data_gen = (list(gen_train_data(df[df['unit_nr']==unit_nr],
sequence_length, columns))
                    for unit_nr in unit_nrs)
        data_array = np.concatenate(list(data_gen)).astype(np.float32)
    return data_array
data_array = gen_data_wrapper(example_df, sequence_length=4,
columns=['X'])
data_array

array([[ [10. ],
        [10.2],
        [10.4],
        [10.6]],

       [ [10.2],
        [10.4],
        [10.6],
        [10.8]],

       [ [20. ],
        [20.2],
        [20.4],
        [20.6]],

       [ [20.2],
        [20.4],
        [20.6],
        [20.8]]], dtype=float32)

def gen_labels(df, sequence_length, label):
    data_matrix = df[label].values
    num_elements = data_matrix.shape[0]

    # -1 because I want to predict the rul of that last row in the

```

```

sequence, not the next row
    return data_matrix[sequence_length-1:num_elements, :]

def gen_label_wrapper(df, sequence_length, label,
unit_nrs=np.array([])):
    if unit_nrs.size <= 0:
        unit_nrs = df['unit_nr'].unique()

    label_gen = [gen_labels(df[df['unit_nr']==unit_nr],
sequence_length, label)
        for unit_nr in unit_nrs]
    label_array = np.concatenate(label_gen).astype(np.float32)
    return label_array
label = gen_labels(example_df[example_df['unit_nr']==1],
sequence_length=4, label=['y'])
label

label_array = gen_label_wrapper(example_df, sequence_length=4,
label=['y'])
label_array

array([[1.2],
       [1.1],
       [2.2],
       [2.1]], dtype=float32)

def gen_test_data(df, sequence_length, columns, mask_value):
    if df.shape[0] < sequence_length:
        data_matrix = np.full(shape=(sequence_length, len(columns)),
fill_value=mask_value) # pad
        idx = data_matrix.shape[0] - df.shape[0]
        data_matrix[idx:,:] = df[columns].values # fill with
available data
    else:
        data_matrix = df[columns].values

    # specifically yield the last possible sequence
    stop = num_elements = data_matrix.shape[0]
    start = stop - sequence_length
    for i in list(range(1)):
        yield data_matrix[start:stop, :]
# notice the masked value I passed is a float,
# it should be the same datatype as the data you're working with
gen = (list(gen_test_data(example_df[example_df['unit_nr']==unit_nr],
sequence_length=6, columns=['X'], mask_value=-99.))
        for unit_nr in example_df['unit_nr'].unique())
next(gen)

[array([[-99. ],
        [ 10. ]],

```

```

        [ 10.2],
        [ 10.4],
        [ 10.6],
        [ 10.8]]])

sequence_length = 20
train['RUL'].clip(upper=125, inplace=True)

X_train_interim = add_operating_condition(train.drop(drop_sensors,
axis=1))
X_test_interim = add_operating_condition(test.drop(drop_sensors,
axis=1))

X_train_interim, X_test_interim = condition_scaler(X_train_interim,
X_test_interim, remaining_sensors)

X_train_interim = exponential_smoothing(X_train_interim,
remaining_sensors, 0, 0.4)
X_test_interim = exponential_smoothing(X_test_interim,
remaining_sensors, 0, 0.4)

# train-val split
gss = GroupShuffleSplit(n_splits=1, train_size=0.80, random_state=42)
for train_unit, val_unit in
gss.split(X_train_interim['unit_nr'].unique(),
groups=X_train_interim['unit_nr'].unique()):
    train_unit = X_train_interim['unit_nr'].unique()[train_unit] #
gss returns indexes and index starts at 1
    val_unit = X_train_interim['unit_nr'].unique()[val_unit]

    train_split_array = gen_data_wrapper(X_train_interim,
sequence_length, remaining_sensors, train_unit)
    train_split_label = gen_label_wrapper(X_train_interim,
sequence_length, ['RUL'], train_unit)

    val_split_array = gen_data_wrapper(X_train_interim,
sequence_length, remaining_sensors, val_unit)
    val_split_label = gen_label_wrapper(X_train_interim,
sequence_length, ['RUL'], val_unit)

# create sequences train, test
train_array = gen_data_wrapper(X_train_interim, sequence_length,
remaining_sensors)
label_array = gen_label_wrapper(X_train_interim, sequence_length,
['RUL'])

test_gen =
(list(gen_test_data(X_test_interim[X_test_interim['unit_nr']==unit_nr]
, sequence_length, remaining_sensors, -99.))
    for unit_nr in X_test_interim['unit_nr'].unique())

```

```
test_array = np.concatenate(list(test_gen)).astype(np.float32)
input_shape=(sequence_length, train_array.shape[2])
```

```
print(train_array.shape)
```

C:\Users\Kamal\AppData\Local\Temp\ipykernel_23140\3985479474.py:4:
FutureWarning: Not prepending group keys to the result index of
transform-like apply. In the future, the group keys will be included
in the index, regardless of whether the applied function returns a
like-indexed object.

To preserve the previous behavior, use

```
>>> .groupby(..., group_keys=False)
```

To adopt the future behavior and silence this warning, use

```
>>> .groupby(..., group_keys=True)
df[sensors] = df.groupby('unit_nr')[sensors].apply(lambda x:
x.ewm(alpha=alpha).mean())
```

C:\Users\Kamal\AppData\Local\Temp\ipykernel_23140\3985479474.py:4:
FutureWarning: Not prepending group keys to the result index of
transform-like apply. In the future, the group keys will be included
in the index, regardless of whether the applied function returns a
like-indexed object.

To preserve the previous behavior, use

```
>>> .groupby(..., group_keys=False)
```

To adopt the future behavior and silence this warning, use

```
>>> .groupby(..., group_keys=True)
df[sensors] = df.groupby('unit_nr')[sensors].apply(lambda x:
x.ewm(alpha=alpha).mean())
```

```
(22820, 20, 14)
```

```
X_train=train_array;
y_train=label_array;
X_val=val_split_array;
y_val=val_split_label;
X_test=test_array;
y_test=y_test;
print(X_train)
```

```
[[[-0.1957505 -0.7036457 -0.7872241 ... -0.89511585  0.47184342
  -0.2703317 ]
 [-0.02314901 -0.5704098 -0.77477324 ... -0.5370391  0.18541567
   0.13753062]
 [-0.28156015 -0.70987767 -0.31913024 ... -0.7142265  -0.19008097
  -0.01937994]
 ...
```

```
[ -0.5920709  -0.5564649  -0.6074026  ...  -0.44581202  0.26368454
 0.39784428]
[ -0.47461858  -0.5725128  -0.5949152  ...  -0.62122846  0.19629976
 0.28012225]
[ -0.43975362  -0.83436024  -0.72333044  ...  -0.7307874  -0.08100169
 0.42912084]]
```

```
[ [ -0.02314901  -0.5704098  -0.77477324  ...  -0.5370391  0.18541567
 0.13753062]
[ -0.28156015  -0.70987767  -0.31913024  ...  -0.7142265  -0.19008097
 -0.01937994]
[ 0.2512429  -0.5470874  -0.7505111  ...  -0.53350365  -0.16414258
 -0.31666037]
...
[ -0.47461858  -0.5725128  -0.5949152  ...  -0.62122846  0.19629976
 0.28012225]
[ -0.43975362  -0.83436024  -0.72333044  ...  -0.7307874  -0.08100169
 0.42912084]
[ -0.1405087  -0.37025115  -0.695096  ...  -0.7951756  -0.15585956
 0.57083505]]
```

```
[ [ -0.28156015  -0.70987767  -0.31913024  ...  -0.7142265  -0.19008097
 -0.01937994]
[ 0.2512429  -0.5470874  -0.7505111  ...  -0.53350365  -0.16414258
 -0.31666037]
[ -0.49950945  -0.280429  -0.7281679  ...  -0.44273242  0.16934638
 0.00761612]
...
[ -0.43975362  -0.83436024  -0.72333044  ...  -0.7307874  -0.08100169
 0.42912084]
[ -0.1405087  -0.37025115  -0.695096  ...  -0.7951756  -0.15585956
 0.57083505]
[ -0.3332497  -0.33664823  -0.72016364  ...  -0.6067915  -0.04255541
 0.23463233]]
```

...

```
[ [ 1.4431524  1.3201514  1.6539075  ...  0.9884535  -1.8254077
 -1.5662149 ]
[ 1.1427828  1.191281  1.6801834  ...  1.1380484  -1.6171088
 -1.5024166 ]
[ 1.6560513  0.96125644  1.8446919  ...  1.4526936  -1.8740114
 -1.5879861 ]
...
[ 2.3249125  1.7007577  1.9618126  ...  1.3625681  -2.043015
 -2.2779903 ]
[ 2.027329  1.5195819  2.189616  ...  1.3721372  -2.009149
 -2.4892445 ]
[ 2.2730138  1.8760715  2.4228036  ...  1.6135205  -2.1792784
 -2.3732708 ]]
```

```

[[ 1.1427828  1.191281  1.6801834 ... 1.1380484 -1.6171088
-1.5024166 ]
[ 1.6560513  0.96125644 1.8446919 ... 1.4526936 -1.8740114
-1.5879861 ]
[ 1.5424378  1.0729072  2.1410275 ... 1.654037 -1.698483
-1.6035374 ]
...
[ 2.027329  1.5195819  2.189616 ... 1.3721372 -2.009149
-2.4892445 ]
[ 2.2730138  1.8760715  2.4228036 ... 1.6135205 -2.1792784
-2.3732708 ]
[ 2.4971085  1.6823083  2.3633585 ... 1.5286937 -2.4105256
-2.2844982 ]]

```

```

[[ 1.6560513  0.96125644 1.8446919 ... 1.4526936 -1.8740114
-1.5879861 ]
[ 1.5424378  1.0729072  2.1410275 ... 1.654037 -1.698483
-1.6035374 ]
[ 1.3069627  1.2095582  2.3576148 ... 1.3168281 -1.6913663
-1.4066046 ]
...
[ 2.2730138  1.8760715  2.4228036 ... 1.6135205 -2.1792784
-2.3732708 ]
[ 2.4971085  1.6823083  2.3633585 ... 1.5286937 -2.4105256
-2.2844982 ]
[ 2.4000597  1.6455269  2.6951518 ... 1.6870807 -2.127297
-2.195497  ]]]

```

```

X_train=train_array;
y_train=label_array;
X_val=val_split_array;
y_val=val_split_label;
X_test=test_array;
y_test=y_test;
print(X_train)
# Create the fitness function for the GA
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

# Define the evaluation function for the fitness function
def evaluate(individual):
    # [toolbox.attr_n_conv_filters, toolbox.attr_conv_activation_func,
    toolbox.attr_lstm_activation_func,
    # toolbox.attr_n_conv_layers_func,
    toolbox.attr_n_lstm_layers_func, toolbox.attr_n_lstm_filters,
    toolbox.attr_lr ]
    # Extract the number of filters and activation function from the
    individual
    try:

```

```

print('*'*15)
print(individual)
activation_function_list = ['relu', 'sigmoid', 'tanh']
lr_list = np.arange(0.0001, 0.1, (0.1-0.0001)/10)
n_conv_filters = individual[0]
conv_activation = activation_function_list[individual[1]]
lstm_activation = activation_function_list[individual[2]]
n_conv_layers = individual[3]
n_lstm_layers = individual[4]
n_lstm_filters = individual[5]
lr = lr_list[individual[6]]

# Create the CNN-LSTM model
model = Sequential()
model.add(Conv1D(filters=n_conv_filters, kernel_size=2,
activation=conv_activation, input_shape=input_shape))
for i in range(n_conv_layers):
    model.add(Conv1D(filters=n_conv_filters, kernel_size=2,
activation=conv_activation))
model.add(Dropout(0))
for i in range(n_lstm_layers):
    return_sequences = True
    if i >= n_lstm_layers-1:
        return_sequences = False
    model.add(LSTM(n_lstm_filters, activation=lstm_activation,
return_sequences=return_sequences))
model.add(Dropout(0.2))
model.add(Dense(200, activation='relu'))
model.add(Dense(1, activation = 'linear'))

print(model.summary())

# for i, layer in enumerate (model.layers):
#     print (i, layer)
#     try:
#         print ("      ",layer.activation)
#     except AttributeError:
#         print('      no activation attribute')

# Compile the model
model.compile(optimizer=Adam(learning_rate=lr),
loss='mean_squared_error')

# Train the model
es = EarlyStopping(monitor='val_loss', patience=10)
model.fit(X_train, y_train, epochs=20, batch_size=32,
validation_data=(X_test, y_test), callbacks=[es], verbose=0)

# Predict on the test set

```

```

    y_pred = model.predict(X_test)

    # Calculate RMSE
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
except:
    print('individual failed !!!')
    rmse = 100000
return rmse,

# Create the toolbox for the GA
toolbox = base.Toolbox()
toolbox.register("attr_n_conv_filters", np.random.randint, 16, 110)
# toolbox.register("attr_conv_activation_func", np.random.choice,
# ['relu', 'sigmoid', 'tanh'])
# toolbox.register("attr_lstm_activation_func", np.random.choice,
# ['relu', 'sigmoid', 'tanh'])
toolbox.register("attr_conv_activation_func", np.random.randint, 0, 3)
toolbox.register("attr_lstm_activation_func", np.random.randint, 0, 3)
toolbox.register("attr_n_conv_layers_func", np.random.randint, 1, 6)
toolbox.register("attr_n_lstm_layers_func", np.random.randint, 1, 6)
# toolbox.register("attr_n_lstm_layers_func", np.random.randint, 2, 4)
toolbox.register("attr_n_lstm_filters", np.random.randint, 16, 110)
# toolbox.register("attr_lr", np.random.uniform, 0.0001, 0.1)
toolbox.register("attr_lr", np.random.randint, 0, 10)

functions = [toolbox.attr_n_conv_filters,
             toolbox.attr_conv_activation_func, toolbox.attr_lstm_activation_func,
             toolbox.attr_n_conv_layers_func,
             toolbox.attr_n_lstm_layers_func, toolbox.attr_n_lstm_filters,
             toolbox.attr_lr ]
toolbox.register("individual", tools.initCycle, creator.Individual,
                functions, n=1)
toolbox.register("population", tools.initRepeat, list,
                toolbox.individual)

# Register the evaluation function and genetic operators
toolbox.register("evaluate", evaluate)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutUniformInt, low=5, up=20,
                indpb=0.1)
toolbox.register("select", tools.selTournament, tournsize=3)

def main():
    # Set the population size and number of generations
    pop_size = 20
    n_generations = 10

    # Create the initial population
    population = toolbox.population(n=pop_size)

```

```

# Run the GA
algorithms.eaSimple(population, toolbox, cxpb=0.6, mutpb=0.3,
ngen=n_generations, verbose=True)

# Get the best individual
best_individual = tools.selBest(population, k=1)[0]
print("Best Individual:", best_individual)

# Evaluate the best individual
rmse = evaluate(best_individual)
print("RMSE for Best Individual:", rmse[0])

if __name__ == "__main__":
    main()

```

```

[[[-0.1957505  -0.7036457  -0.7872241  ... -0.89511585  0.47184342
  -0.2703317 ]
 [-0.02314901 -0.5704098  -0.77477324 ... -0.5370391  0.18541567
  0.13753062]
 [-0.28156015 -0.70987767 -0.31913024 ... -0.7142265  -0.19008097
  -0.01937994]
 ...
 [-0.5920709  -0.5564649  -0.6074026  ... -0.44581202  0.26368454
  0.39784428]
 [-0.47461858 -0.5725128  -0.5949152  ... -0.62122846  0.19629976
  0.28012225]
 [-0.43975362 -0.83436024 -0.72333044 ... -0.7307874  -0.08100169
  0.42912084]]

[[-0.02314901 -0.5704098  -0.77477324 ... -0.5370391  0.18541567
  0.13753062]
 [-0.28156015 -0.70987767 -0.31913024 ... -0.7142265  -0.19008097
  -0.01937994]
 [ 0.2512429  -0.5470874  -0.7505111  ... -0.53350365 -0.16414258
  -0.31666037]
 ...
 [-0.47461858 -0.5725128  -0.5949152  ... -0.62122846  0.19629976
  0.28012225]
 [-0.43975362 -0.83436024 -0.72333044 ... -0.7307874  -0.08100169
  0.42912084]
 [-0.1405087  -0.37025115 -0.695096   ... -0.7951756  -0.15585956
  0.57083505]]

[[[-0.28156015 -0.70987767 -0.31913024 ... -0.7142265  -0.19008097
  -0.01937994]
 [ 0.2512429  -0.5470874  -0.7505111  ... -0.53350365 -0.16414258
  -0.31666037]
 [-0.49950945 -0.280429   -0.7281679  ... -0.44273242  0.16934638
  0.00761612]]

```

```
...
[ -0.43975362 -0.83436024 -0.72333044 ... -0.7307874 -0.08100169
  0.42912084 ]
[ -0.1405087 -0.37025115 -0.695096 ... -0.7951756 -0.15585956
  0.57083505 ]
[ -0.3332497 -0.33664823 -0.72016364 ... -0.6067915 -0.04255541
  0.23463233 ]
```

...

```
[ [ 1.4431524 1.3201514 1.6539075 ... 0.9884535 -1.8254077
    -1.5662149 ]
  [ 1.1427828 1.191281 1.6801834 ... 1.1380484 -1.6171088
    -1.5024166 ]
  [ 1.6560513 0.96125644 1.8446919 ... 1.4526936 -1.8740114
    -1.5879861 ]
```

...

```
[ 2.3249125 1.7007577 1.9618126 ... 1.3625681 -2.043015
  -2.2779903 ]
[ 2.027329 1.5195819 2.189616 ... 1.3721372 -2.009149
  -2.4892445 ]
[ 2.2730138 1.8760715 2.4228036 ... 1.6135205 -2.1792784
  -2.3732708 ]
```

```
[ [ 1.1427828 1.191281 1.6801834 ... 1.1380484 -1.6171088
    -1.5024166 ]
  [ 1.6560513 0.96125644 1.8446919 ... 1.4526936 -1.8740114
    -1.5879861 ]
  [ 1.5424378 1.0729072 2.1410275 ... 1.654037 -1.698483
    -1.6035374 ]
```

...

```
[ 2.027329 1.5195819 2.189616 ... 1.3721372 -2.009149
  -2.4892445 ]
[ 2.2730138 1.8760715 2.4228036 ... 1.6135205 -2.1792784
  -2.3732708 ]
[ 2.4971085 1.6823083 2.3633585 ... 1.5286937 -2.4105256
  -2.2844982 ]
```

```
[ [ 1.6560513 0.96125644 1.8446919 ... 1.4526936 -1.8740114
    -1.5879861 ]
  [ 1.5424378 1.0729072 2.1410275 ... 1.654037 -1.698483
    -1.6035374 ]
  [ 1.3069627 1.2095582 2.3576148 ... 1.3168281 -1.6913663
    -1.4066046 ]
```

...

```
[ 2.2730138 1.8760715 2.4228036 ... 1.6135205 -2.1792784
  -2.3732708 ]
[ 2.4971085 1.6823083 2.3633585 ... 1.5286937 -2.4105256
  -2.2844982 ]
[ 2.4000597 1.6455269 2.6951518 ... 1.6870807 -2.127297
```

```
-2.195497  ]]]
*****
[67, 0, 2, 3, 5, 36, 6]
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 19, 67)	1943
conv1d_1 (Conv1D)	(None, 18, 67)	9045
conv1d_2 (Conv1D)	(None, 17, 67)	9045
conv1d_3 (Conv1D)	(None, 16, 67)	9045
dropout (Dropout)	(None, 16, 67)	0
lstm (LSTM)	(None, 16, 36)	14976
lstm_1 (LSTM)	(None, 16, 36)	10512
lstm_2 (LSTM)	(None, 16, 36)	10512
lstm_3 (LSTM)	(None, 16, 36)	10512
lstm_4 (LSTM)	(None, 36)	10512
dropout_1 (Dropout)	(None, 36)	0
dense (Dense)	(None, 200)	7400
dense_1 (Dense)	(None, 1)	201

```
=====
Total params: 93703 (366.03 KB)
Trainable params: 93703 (366.03 KB)
Non-trainable params: 0 (0.00 Byte)
```

```
None
individual failed !!!
*****
[98, 2, 2, 3, 5, 39, 2]
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv1d_4 (Conv1D)	(None, 19, 98)	2842
conv1d_5 (Conv1D)	(None, 18, 98)	19306

conv1d_6 (Conv1D)	(None, 17, 98)	19306
conv1d_7 (Conv1D)	(None, 16, 98)	19306
dropout_2 (Dropout)	(None, 16, 98)	0
lstm_5 (LSTM)	(None, 16, 39)	21528
lstm_6 (LSTM)	(None, 16, 39)	12324
lstm_7 (LSTM)	(None, 16, 39)	12324
lstm_8 (LSTM)	(None, 16, 39)	12324
lstm_9 (LSTM)	(None, 39)	12324
dropout_3 (Dropout)	(None, 39)	0
dense_2 (Dense)	(None, 200)	8000
dense_3 (Dense)	(None, 1)	201
=====		
Total params: 139785 (546.04 KB)		
Trainable params: 139785 (546.04 KB)		
Non-trainable params: 0 (0.00 Byte)		
None		